

DID 신원인증 모듈 개발

안전한 메타버스 환경을 위한 사용자 인증 및 프라이버시 보호 기술개발

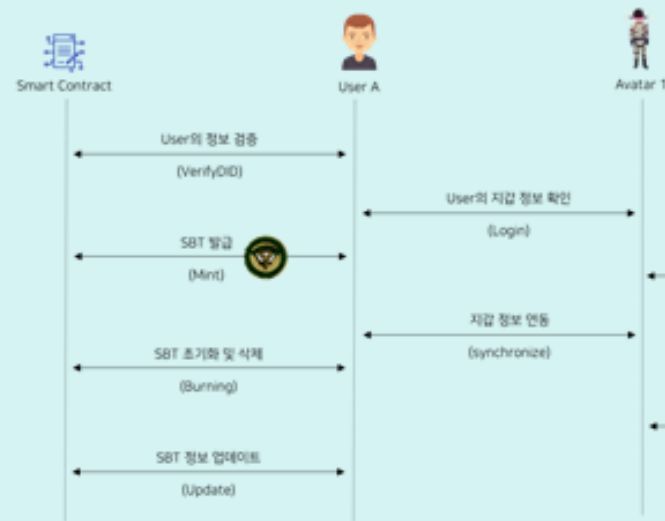
#충남대학교 정보보호연구실 #함윤식

Please enter a title

1차년도

- 아바타 인증과 권한 관리를 위한 신원 체계 구축

- 메타버스 아바타 신원 규격 정의



ERC-7574 : Authentication SBT using Credential

- 아바타 인증을 위한 Presentation struct 정의
- SBT 토큰 발행 및 검증 스마트 컨트랙트 포맷 정의

```

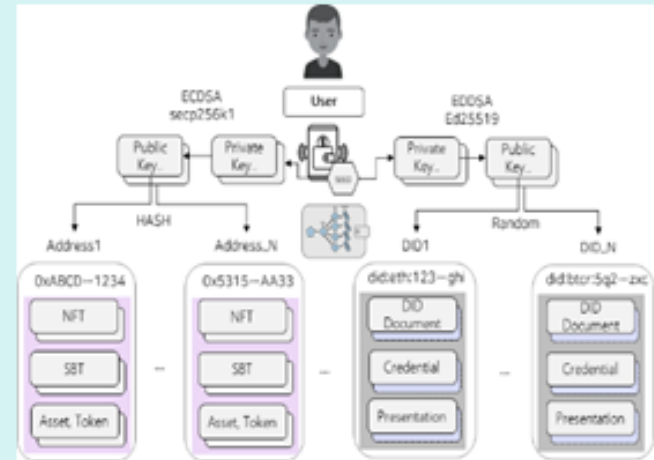
// SPDX-License-Identifier: CCB-1.0
pragma solidity ^0.8.0; interface IERC5192 {
    /// @ Function to verify the user's proof.
    /// @ Generated through ZkSnarks, required param
    /// @ Other information such as issuer/holder
    function verify(Proof memory proof, uint[2] memory info) returns (bool);
    /// @ Only those who have completed verifyProof
    /// @ Only the service provider can call the createSBT
    /// @ Issues SBT to authenticated users, with
    function createSBT(address user, string memory tokenID) public onlyOwner returns (uint);
    /// @ Only the service provider can call
    /// @ Issuing an SBT allows cancellation
    function updateTokenID(address user, string memory tokenID) public onlyOwner returns (uint);
    /// @ Added cannot transfer functionality
    function safeTransferFrom(address _from, address _to, uint _tokenId) public onlyOwner returns (bool);
}
    
```

2차년도

- 아바타 신원 및 SBT를 위한 통합 지갑 개발

- DID와 SBT를 관리할 수 있는 통합 지갑

- W3C의 Universal Wallet 2020 요구사항 반영



- 하나의 마스터 시드(SEED)를 통한 HD 통합 지갑 개발

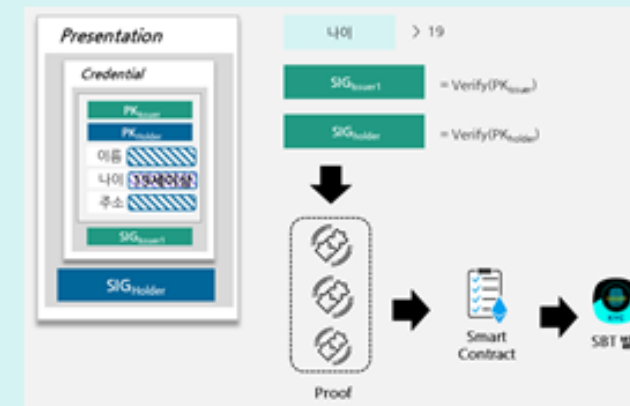
- DID를 관리 하기 위해 EDDSA의 Ed25519 curve를 활용하여 PK 생성, Credential 보관

- SBT를 관리하기 위해 ECDSA의 secp256k1 curve를 활용하여 PK 생성, NFT 및 SBT 보관

3차년도

- SBT 발급 및 DID 검증 스마트 컨트랙트 개발

- DID 검증과 SBT 발급을 스마트 컨트랙트를 통해 진행



- 사용자의 개인정보 보호를 위해 영지식 증명이 적용된 스마트 컨트랙트 개발

- ZK-snarks 기반 영지식증명 알고리즘인 Pinocchio를 활용, 기존 Zokrates 도구를 변형하여 검증 스마트 컨트랙트 생성

- 사용자의 DID를 입력으로 받아, 사용자의 정보에 대한 Proof를 생성한 이후 스마트 컨트랙트를 통해 검증 하여 SBT 발급

4차년도

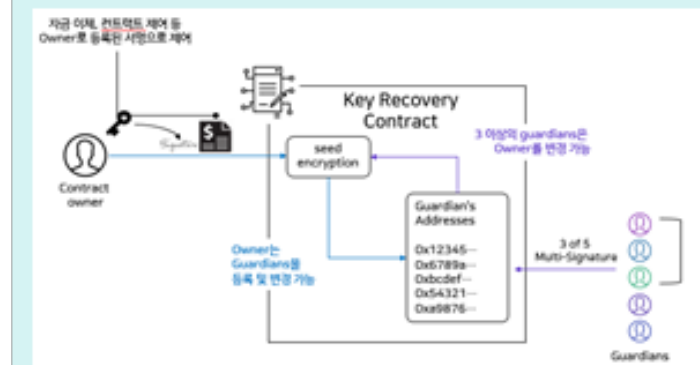
- 메타버스에서의 아바타 권한 관리 실증 서비스 구현

- 메타버스 아바타 매핑(실증서비스)

- 신원 인증 체계, 통합 관리지갑, 스마트 컨트랙트 생성 도구를 이용한 아바타 신원 인증 서비스 구현

- SBT 안전성 평가, 스마트 컨트랙트 취약점 및 실행 비용 최적화 방안 연구

- 사회적 복구(Social Recovery)활용 방안



- 사용자가 SBT 및 DID가 들어있는 지갑의 개인키를 잃어버렸을 경우, 키 변경 스마트 컨트랙트를 활용하여 새로 생성된 키로 변경함으로써 지갑의 소유권 복구가 가능

DID & SBT 기반 신원인증 지갑 개발



성과

	SCI(E)	특허출원	특허등록	기술이전
1차년도	0/0	1/1	0/0	
2차년도	0/1	1/1	0/0	1/1(1,000)
3차년도	0/	1/	0/	
4차년도	0/	1(국제특허)/	0/	1/(1,000)

- SCI(E) 1편
 - 영지식 증명 개발 도구 관련
- 기술이전
 - DID&SBT 지갑 관련(1,000만)
- 특허 출원 1건
 - DID 검증 방안

KSII TRANSACTIONS ON INTERNET AND INFORMATION SYSTEMS VOL. 18, NO. 8, Aug. 2024
Copyright © 2024 KSII

2417

Privacy-preserving credential smart contracts using Zokrates

Geunyoung Kim, Yunsik Ham and Jaechool Ryou*
Department of Computer Science, Chungnam National University
Daejeon, South Korea
[e-mail: gykim@cnu.ac.kr]
*Corresponding author: Jaechool Ryou
Received March 31, 2024; accepted June 3, 2024;
published August 31, 2024

Abstract

The need for secure user authentication in blockchain-based applications has been growing with the increased adoption of Decentralized Identity (DID) credentials in blockchain. Zokrates, a tool designed to protect user privacy within smart contracts, had a limitation in that it could not accept authenticated user information such as credentials, only allowing the use of manually inputted data. In this paper, we propose a smart contract system that securely validates DID credentials to overcome the limitations of traditional centralized authentication systems. This system ensures the safe identification of users within blockchain-based applications by authenticating their identities in a trusted manner within the blockchain. As the demand for user authentication in blockchain rises, this paper emphasizes the significance of a blockchain-based identity verification system that guarantees both privacy and security. Leveraging the Zero-Knowledge Proof method and utilizing the Zokrates tool, this innovative approach aims to provide solutions for the digital identity verification process, thereby expanding the scope of blockchain technology applications. Moreover, we also provide a CLI for each entity. We help anyone who wants to authenticate their identity using the tool to safely verify it on-chain.

Keywords: Blockchain, Privacy-preserving, Credential, Zero-knowledge proof

기 술 이 전 계 약 서

계약기술 : DID 및 SBT 지갑 기술 통합 관리 기술



2024. 10. 25.

양 도 인 : 충남대학교 산학협력단

단장 임남형

양 수 인 : ㈜루테스

대표 김기수

출 원 번 호 통 지 서

출 원 일 자 2024.12.27

특 기 사 항 심사청구(유) 공개신청(무) 참조번호(DP241038)

출 원 번 호 10-2024-0199092 (접수번호 1-1-2024-1452172-13)
(DAS접근코드 B9C0)

출 원 인 명 칭 충남대학교산학협력단(2-2004-008410-4)

대 리 인 성 명 특허법인위더피플(9-2019-100141-2)

발 명 자 성 명 류재철 함윤식 홍지나

발 명의 명 칭 스마트 계약을 이용한 분산 식별자 기반의 인증 및 검증 방법

특 허 청 장

<< 안내 >>

1. 귀하의 출원은 위와 같이 정상적으로 접수되었으며, 이후의 심사 진행상황은 출원번호를 이용하여 특허로 홈페이지(www.patent.go.kr)에서 확인하실 수 있습니다.

2. 출원에 따른 수수료는 접수일로부터 다음날까지 통보된 납입영수증에 성명, 납부자번호 등을 기재하여 가까운 은행 또는 우체국에 납부하여야 합니다.

※ 납부자번호 : 0131(기관코드) + 접수번호

3. 귀하의 주소, 연락처 등의 변경사항이 있을 경우, 즉시 (특허고객번호 정보변경(경정), 정정신고서)를 제출하여 출원 이후의 각종 통지서를 정상적으로 받을 수 있습니다.

4. 기타 심사 절차(제도)에 관한 사항은 특허청 홈페이지를 참고하시거나 특허고객상담센터(☎ 1544-8080)에 문의하여 주시기 바랍니다.

※ 심사제도 안내 : https://www.kipo.go.kr/지식재산제도

4

W3C Verifiable Credentials Data Model v2.0

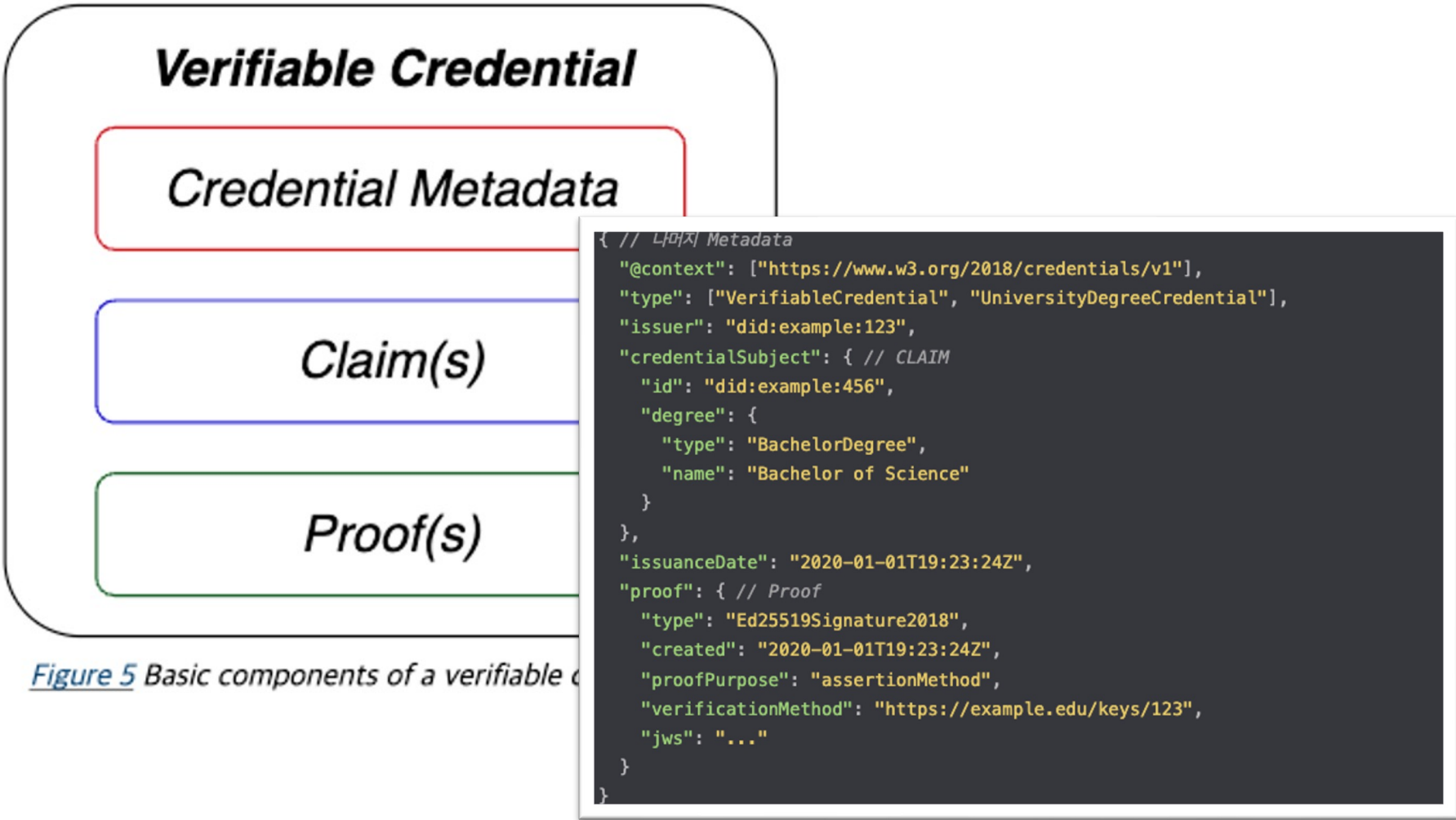


Figure 5 Basic components of a verifiable credential

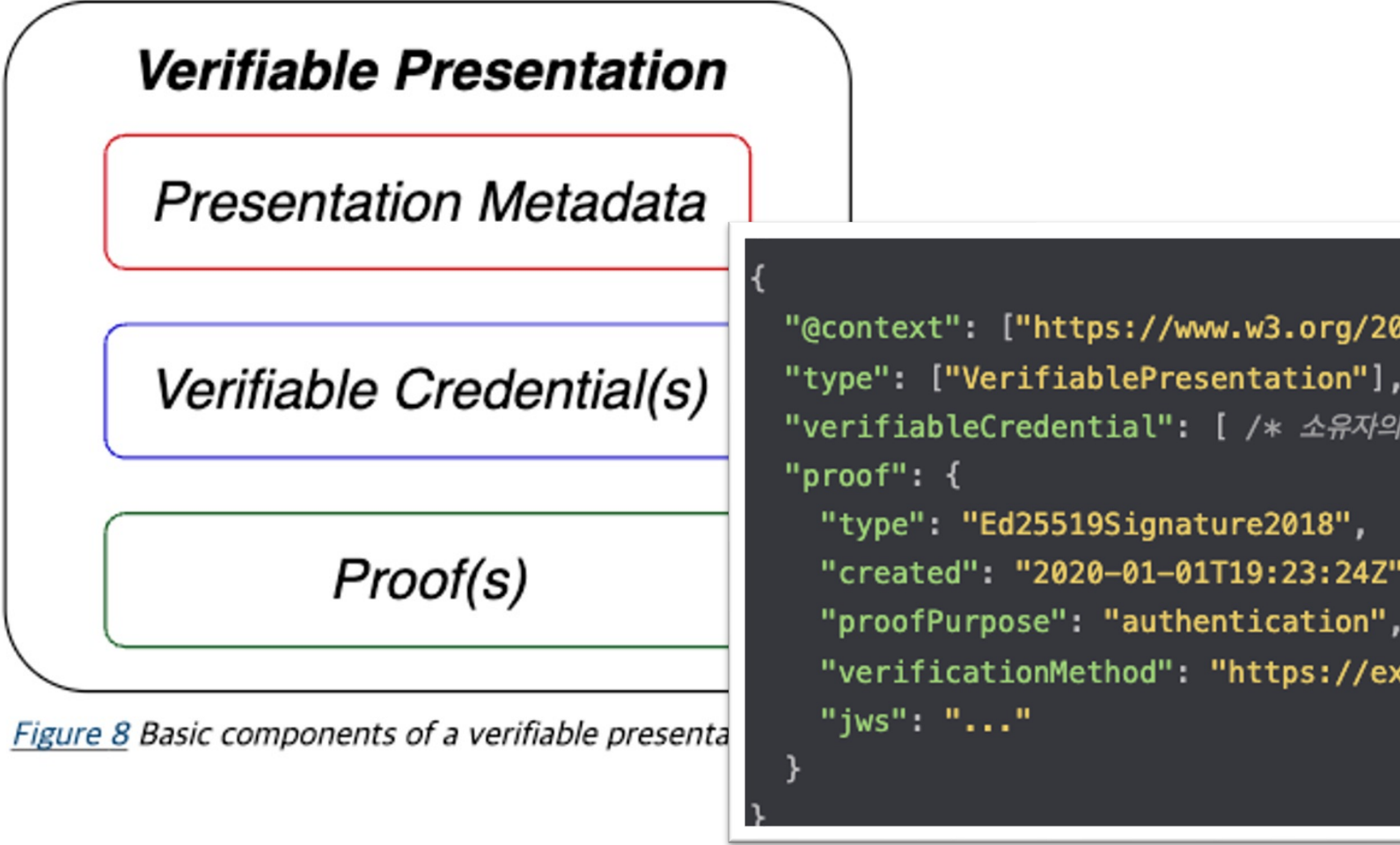


Figure 8 Basic components of a verifiable presentation

W3C Verifiable Credentials Data Model v2.0: 인터넷 상에서 검증 가능한 자격증명을 표현하는 방법을 표준화한 문서.
검증 가능한 자격 증명(VC), 프레젠테이션(VP), 검증 체계
선택적 공개(Selective Disclosure)와 비연결성(Unlinkability)을 포함한 프라이버시 보호 메커니즘을 포함

#01
Background

W3C Verifiable Credentials Data Model v2.0

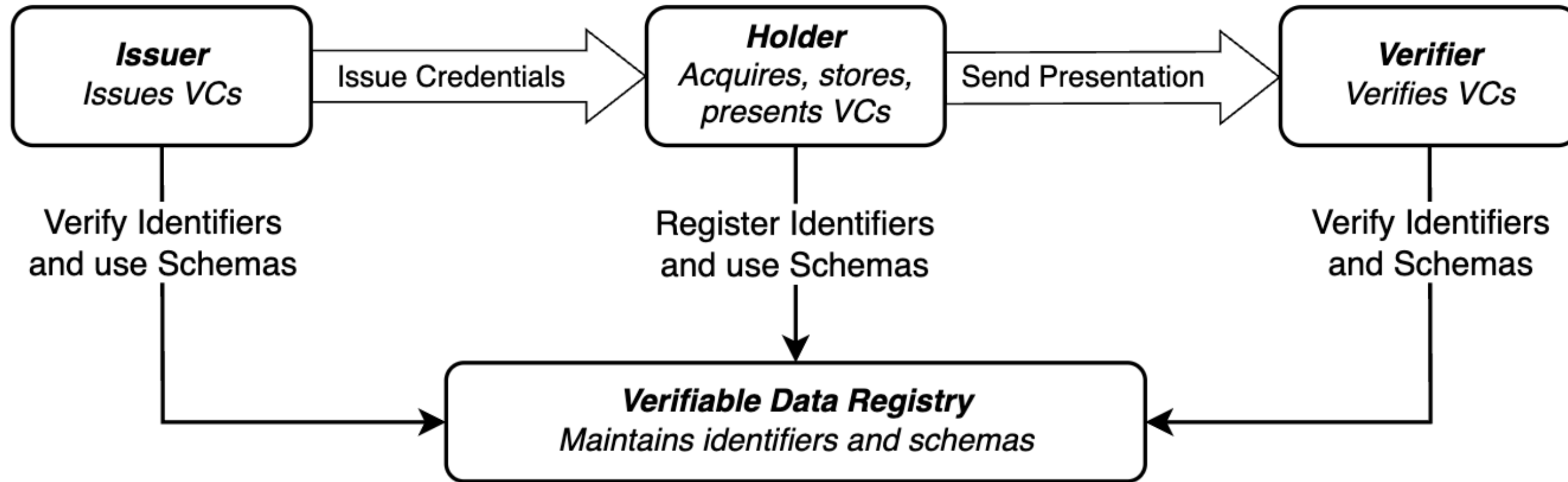


Figure 1 The roles and information flows forming the basis for this specification.

Issuer - Holder에게 VC(Verifiable Credential) 발급

Holder - Verifiable Data Registry를 참조하여 Identifier를 등록하고 VC(Verifiable Credential)를 관리

Verifier - Holder로부터 받은 VP(Verifiable Presentation)을 검증

W3C Verifiable Credentials Data Model v2.0

Securing Mechanisms

포장 증명(Enveloping Proof): 데이터 모델 외부에 래핑된 증명

- JSON Web Signature (JWS), Selective Disclosure JWT (SD-JWT), CBOR Object Signing and Encryption (COSE)
- 신뢰할 수 있는 공개 키와 같은 외부 정보를 필요로 할 수 있음.

EXAMPLE 18: Basic structure of a presentation

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "type": ["VerifiablePresentation", "ExamplePresentation"],
  "verifiableCredential": [{
    "@context": "https://www.w3.org/ns/credentials/v2",
    "id": "data:application/vc+sd-jwt,QzVjV...RMjU",
    "type": "EnvelopedVerifiableCredential"
  }]
}
```

임베디드 증명(Embedded Proof): 데이터 모델 내에 포함된 증명

- Data Integrity Proof
- 외부 참조 없이 데이터 모델 내부의 정보만으로 검증 가능

EXAMPLE 15: A verifiable credential using an embedded proof

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "http://example.gov/credentials/3732",
  "type": ["VerifiableCredential", "ExampleDegreeCredential"],
  "issuer": "did:example:6fb1f712ebe12c27cc26eebfe11",
  "validFrom": "2010-01-01T19:23:24Z",
  "credentialSubject": {
    "id": "https://subject.example/subject/3921",
    "degree": {
      "type": "ExampleBachelorDegree",
      "name": "Bachelor of Science and Arts"
    }
  },
  "proof": {
    "type": "DataIntegrityProof",
    "cryptosuite": "eddsa-rdfc-2022",
    "created": "2021-11-13T18:19:39Z",
    "verificationMethod": "https://university.example/issuers/14#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z58DAfFa9SkqZMVPxAQp...jQCrFPP2oumHKtz"
  }
}
```

W3C Verifiable Credentials Data Model v2.0

Verifiable Presentations

포장된 검증 가능한 자격증명 (Enveloped Verifiable Credentials)

EXAMPLE 19: Basic structure of an enveloped verifiable presentation

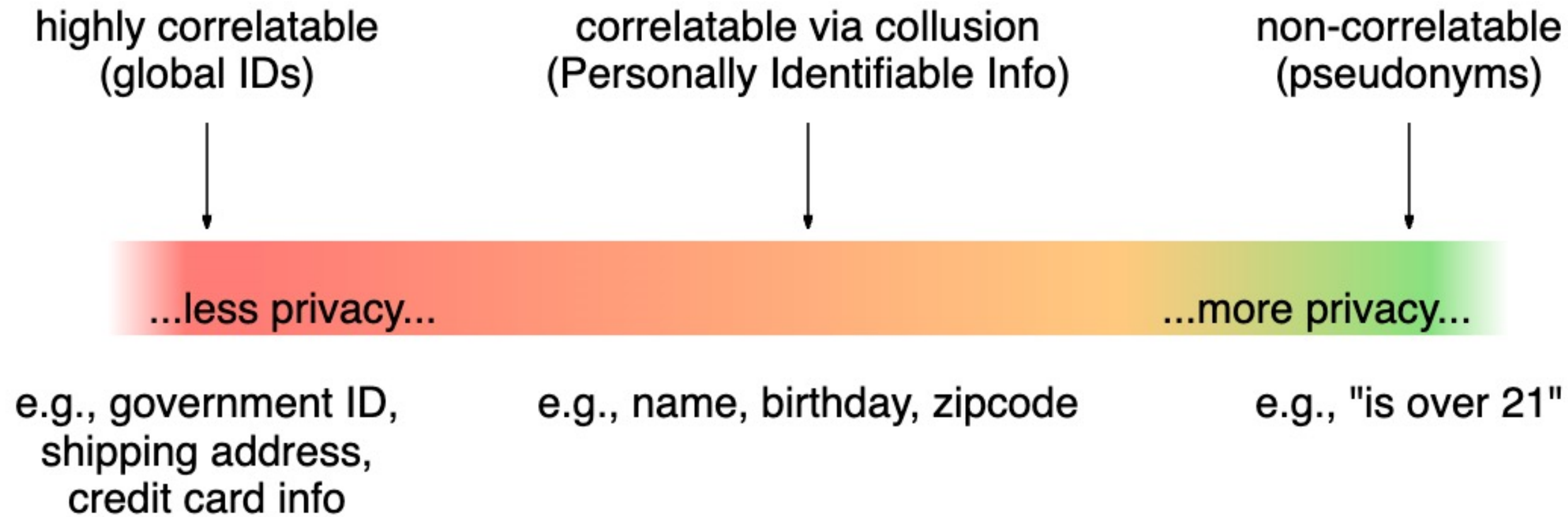
```
{
  "@context": "https://www.w3.org/ns/credentials/v2",
  "id": "data:application/vp+jwt,eyJraWQiO...zhwGfQ",
  "type": "EnvelopedVerifiablePresentation"
}
```

선택적 공개를 위한 ZKP 및 파생된 자격증명(Derived Credentials)

EXAMPLE 21: A verifiable presentation, secured with an embedded Data Integrity proof, with a self-asserted verifiable credential about the verifiable presentation

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "type": ["VerifiablePresentation", "ExamplePresentation"],
  "id": "urn:uuid:313801ba-24b7-11ee-be02-ff560265cf9b",
  "holder": "did:example:12345678",
  "verifiableCredential": [{
    "@context": [
      "https://www.w3.org/ns/credentials/v2",
      "https://www.w3.org/ns/credentials/examples/v2"
    ],
    "type": ["VerifiableCredential", "ExampleAssertCredential"],
    "issuer": "did:example:12345678",
    "credentialSubject": {
      "id": "urn:uuid:313801ba-24b7-11ee-be02-ff560265cf9b",
      "assertion": "This VP is submitted by the subject as evidence of a legal right to drive"
    },
    "proof": { ... }
  }],
  "proof": { ... }
}
```

Privacy Issues in DID Authentication



사용 사례에 따라 익명성부터 완전 식별까지 다양한 프라이버시 요구
데이터 상관성과 PII(Personally Identifiable Information) 노출 위험이 존재

-> 선택적 공개, 가명 식별자, 비연결 암호화 기술 등으로 데이터 최소화와 상관 위험 완화

#02
ZKP Authentication

Privacy Issues in DID Authentication

Unlinkable Proof: 사용자가 동일한 신원으로 여러 인증 요청을 처리하더라도, 요청 간에 연결성을 추적할 수 없도록 설계된 인증 방법

Selective Disclosure: 사용자가 신원 정보 중 특정 부분만 선택적으로 공개하여, 필요한 정보만 제공하고 나머지 데이터는 숨길 수 있는 기술

Ex) bbs-signatures, zk-snark ...

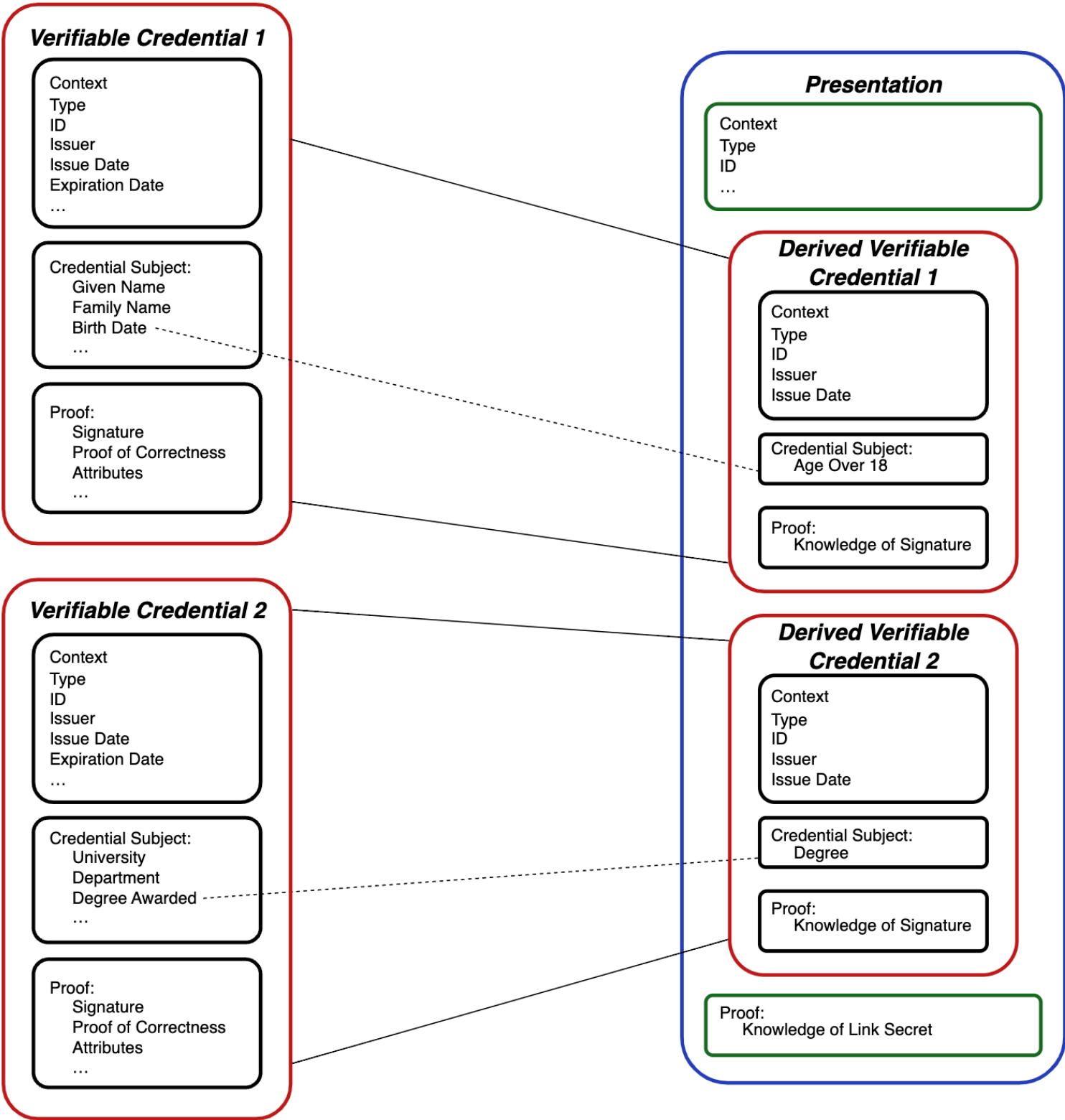


Figure 12 A visual example of the relationship between credentials and derived credentials in a ZKP presentation.

Data Integrity BBS Cryptosuites

BBS Signature scheme, a secure, multi-message digital signature protocol, supporting proving knowledge of a signature while selectively disclosing any subset of the signed messages.

BLS12-381 Curve 기반, Pairing Base Signature, No Setup
Achieving Unlinkable Data Integrity with Pairing-based Cryptography

Element	Size Equation
Private Key	F
Public Key	G2
Signature	G1 + 2*F
Proof	5*G1 + (4 + no_of_hidden_messages)*F

F: A field element.
G1: A point in the field of G1.
G2: A point in the field of G2.
no_of_hidden_messages: The number of hidden messages in the proof.

Signer
(1) Sign

(2) Send signature + msgs ↓

Holder/Prover
(3) ProofGen

(4) Send proof + disclosed msgs ↓

Verifier
(5) ProofVerify

Basic diagram of BBS Scheme

! Review

Standards Track: Core

EIP-2537: Precompile for BLS12-381 curve operations

Adds operation on BLS12-381 curve as a precompile in a set necessary to efficiently perform operations such as BLS signature verification.

Authors Alex Vlasov (@shamatar), Kelly Olson (@ineffectualproperty), Alex Stokes (@ralexstokes), Antonio Sanso (@asanso)

Created 2020-02-21



This EIP is in the process of being peer-reviewed. If you are interested in this EIP, please participate using this discussion link.

Table of Contents

- Abstract
- Motivation
- Specification
 - Constants
 - Curve parameters
 - Fields and Groups
 - Fine points and encoding of base elements
 - ABI for operations
 - Gas burning on error
 - DDoS protection
 - Gas schedule
- Rationale

Comparison of Identity Solutions

요소	uPort	Sovrin	TheKey	SecureKey
네트워크 유형	퍼미션리스	퍼미션드	퍼미션드	퍼미션드
스토리지 위치	- 신원 데이터: 사용자의 장치 - 공개 데이터: 이더리움	- 퍼블릭 데이터: 블록체인 - 민감 데이터: 오프체인(암호화된 형태로)	- 퍼블릭 데이터: 제한적 블록체인 - 민감 데이터: 오프체인	- 민감 데이터: 오프체인(암호화) - 기관 간 데이터 공유
거버넌스 모델	탈중앙화 (이더리움 네트워크)	Sovrin 재단 (중앙화된 거버넌스)	TheKey 팀 및 파트너들 (중앙화)	캐나다 주요 은행 및 파트너 (협력 기반 거버넌스)
읽기 권한	- 공개 데이터: 누구나 가능 - 민감 데이터: 사용자 제어	- 공개 데이터: 일부 일반 유저 가능 - 민감 데이터: 허가 필요	제한적 (허가된 사용자만)	제한적 (기관 또는 사용자)
쓰기 권한	누구나 가능	허가된 노드만 가능	허가된 노드만 가능	허가된 노드만 가능
사용 사례	개인 신원 관리 탈중앙화 인증	탈중앙화 신원 관리 디지털 신원 제공	정부 주도 신원 인증 건강 보험, 복지 등	디지털 신원 인증 금융 데이터 공유
데이터 주권	사용자에게 귀속	사용자에게 귀속 (거버넌스에 따라 제한)	국가 또는 기관에 귀속	사용자와 기관 간 협력에 귀속
주요 기술	이더리움 스마트 컨트랙트	Hyperledger Indy	퍼미션드 블록체인	Federated Identity 기술

다양한 DID System을 통합할 수 있는 ZKP 인증 스킴 정의 & EVM Target 인증 모듈 개발

Babyjubjub & SMT Verifiable Credential Scheme

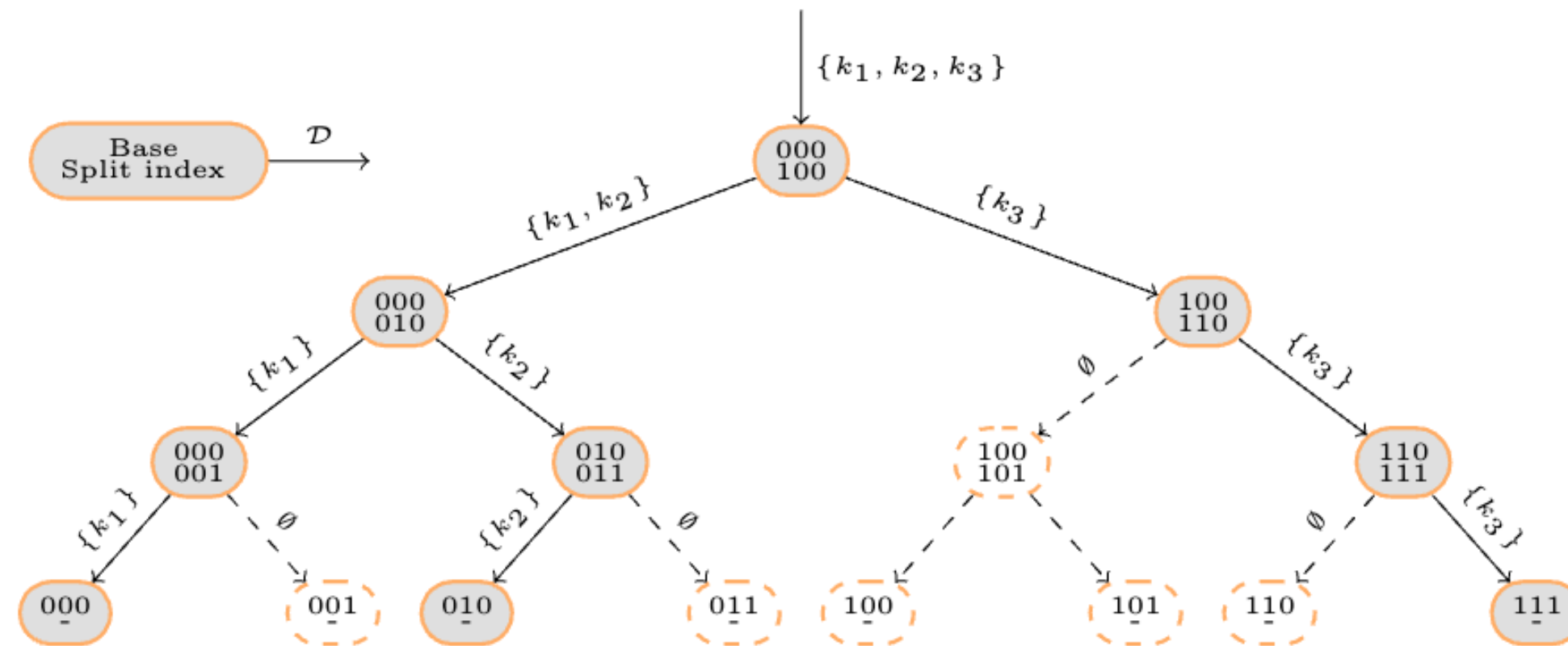


Fig. 3: An illustration of a recursive traversal to obtain the root digest; $k_1 = 000$, $k_2 = 010$, and $k_3 = 111$.

DID의 Claim(Credential Subject)를 SMT 트리에 고정으로 지정된 Key로 저장 Ex) id: 0(000), name:1(001), age:2(010), ...

- SMT Root를 msg로 Babyjubjub Curve 기반 Sign -> Data Integrity 지원

- VC 내부 SMT Root를 통해 Claim 속성의 포함/비포함에 대한 ZKP 생성 지원 -> Selective Disclosure 및 자격 검증 지원

EXAMPLE 4: Use of the type property

Credential	ecdsa	eddsa	ecdsa-sd	bbs	jose	cose	sd-jwt
------------	-------	-------	----------	-----	------	------	--------

application/vc

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://www.w3.org/ns/credentials/examples/v2"
  ],
  "id": "http://university.example/credentials/3732",
  "type": [
    "VerifiableCredential",
    "ExampleDegreeCredential"
  ],
  "issuer": "https://university.example/issuers/565049",
  "validFrom": "2010-01-01T00:00:00Z",
  "credentialSubject": {
    "id": "did:example:ebfeb1f712ebc6f1c276e12ec21",
    "degree": {
      "type": "ExampleBachelorDegree",
      "name": "Bachelor of Science and Arts"
    }
  },
  "proof": {
    "type": "DataIntegrityProof",
    "verificationMethod": "did:key:zUC7GsSgQ4NbriEoGBNeyC7WnG7c5McQMJryqVAK
J2nVgJWvWd3Vv4Ab81iJdDd5WBqSPDxwNjUedgkY2GbsJ5LcsFA5FM6Ub2eoUAnN9NbfZP89UYE
gvfrdurZofw2qfPAFzq6",
    "cryptosuite": "bbs-2023",
    "proofPurpose": "assertionMethod",
    "proofValue": "u2V0ChVhQuEJjhNMn6_EZyF2s6UBsw7Tv2XZZk377KYnEtEidifQKjnD
f371omrp1KeuelAZ6VUX4Qg010TKcikCuL06E4a8kMZ0CA4mNDF9yd1Ez1kRYQGoquS0xvf4Mnm
ZoNkbVv5w8HN4gxs_sTcqvh3oJhCb2S1r7HKFDPyyrvPGqNF8bjgNELvoom0jpbD9JEvaGI1pYY
K5F3ceZ2_Sp8-fh6XzU7h6IO_rR82lnfEmpsNzTVnuS2lfrkjyvNUrJik-kyoecigyt0PDfU43v
VYs_mzcYjx2PjkSg_6KB0UFEXB1G6LE7Wxo4xd0nYmErU2Rih3SB6VggQ84wP_7ZkhhzqzvWHa1
54nBLtixNrTLmJwLHkNBb-HWBZy9pc3N1ZXI"
  }
}
```

```

  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://www.example.org/examples/v1"
  ],
  "id": "http://chungnam.ac.kr/credentials/3732",
  "type": [
    "VerifiableCredential",
    "AlumniCredential"
  ],
  "issuer": {
    "id": "https://infosec.chungnam.ac.kr",
    "name": "Chungnam National University Information Security Lab",
    "publicKey": {
      "Ax": "13277427435165878497778222415993513565335242147425444199013288855685581939618",
      "Ay": "13622229784656158136036771217484571176836296686641868549125388198837476602820"
    }
  },
  "issuanceDate": "2024-02-11T09:30:24Z",
  "credentialSubject": {
    "id": "did:example:abcdef1234567890",
    "name": "ham3798",
    "age": 25,
    "studentNumber": "201902769",
    "alumniOf": {
      "id": "did:example:c34fb4561237890",
      "name": "Chungnam National University",
      "department": "Information Security Lab"
    }
  },
  "proof": {
    "type": "BabyJubJubSMTSignature2024",
    "created": "2024-10-08T05:59:43.348Z",
    "proofPurpose": "zkProof",
    "verificationMethod": "https://infosec.chungnam.ac.kr",
    "merkleRoot": "176,113,248,159,95,186,151,221,221,94,150,40,177,156,132,61,209,46,57,221",
    "signature": {
      "R8x": "14843327923499602530760840822364653819560149483324056861844414827173045711128",
      "R8y": "16624741311753369209800270717954129820251768033844592664257222687029635194829",
      "S": "2003340101055013093731532733602979271886157286292523269931749695412007991258"
    }
  }
}
```

Generating and verifying ZK-SNARK proofs

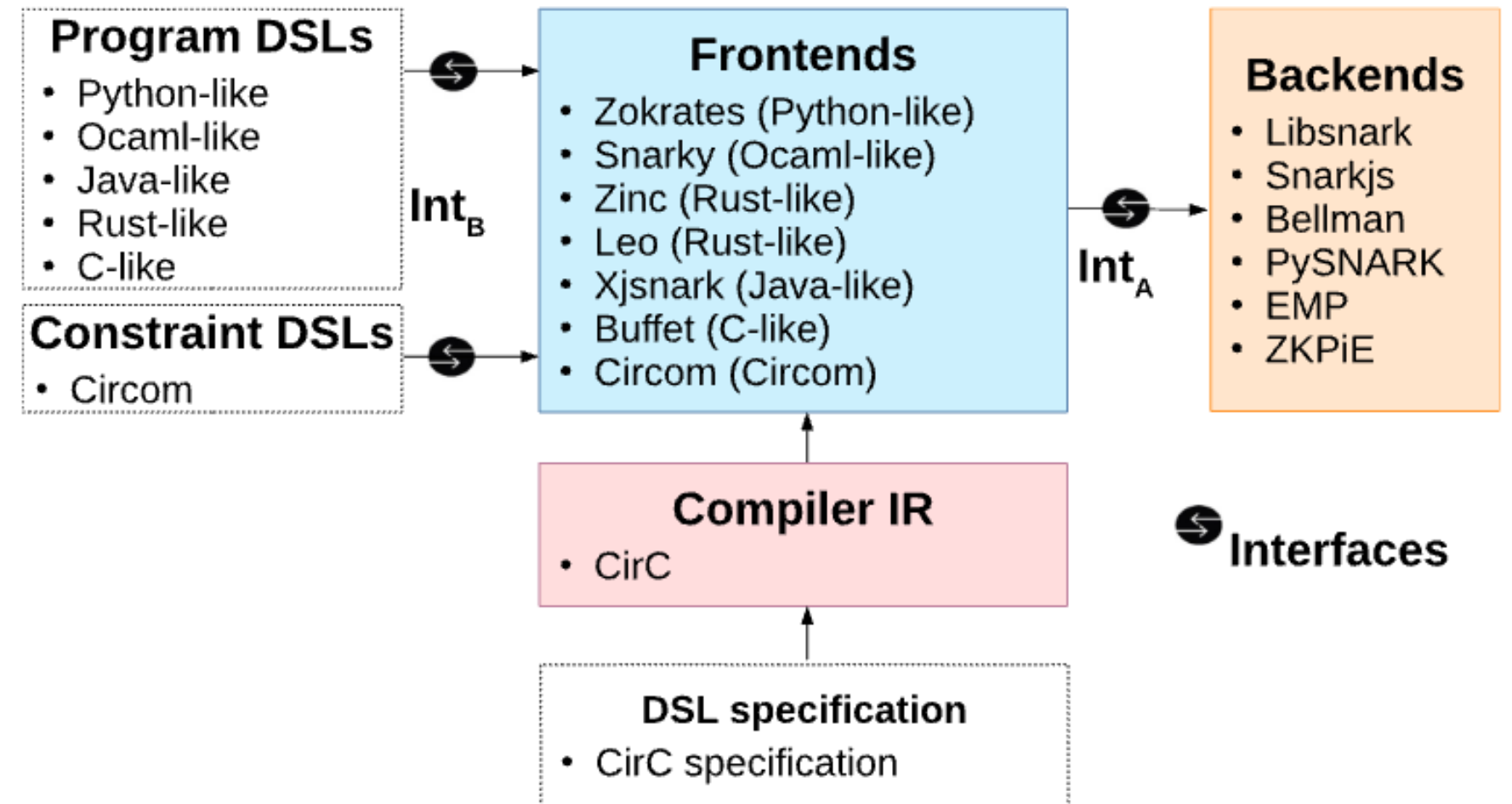
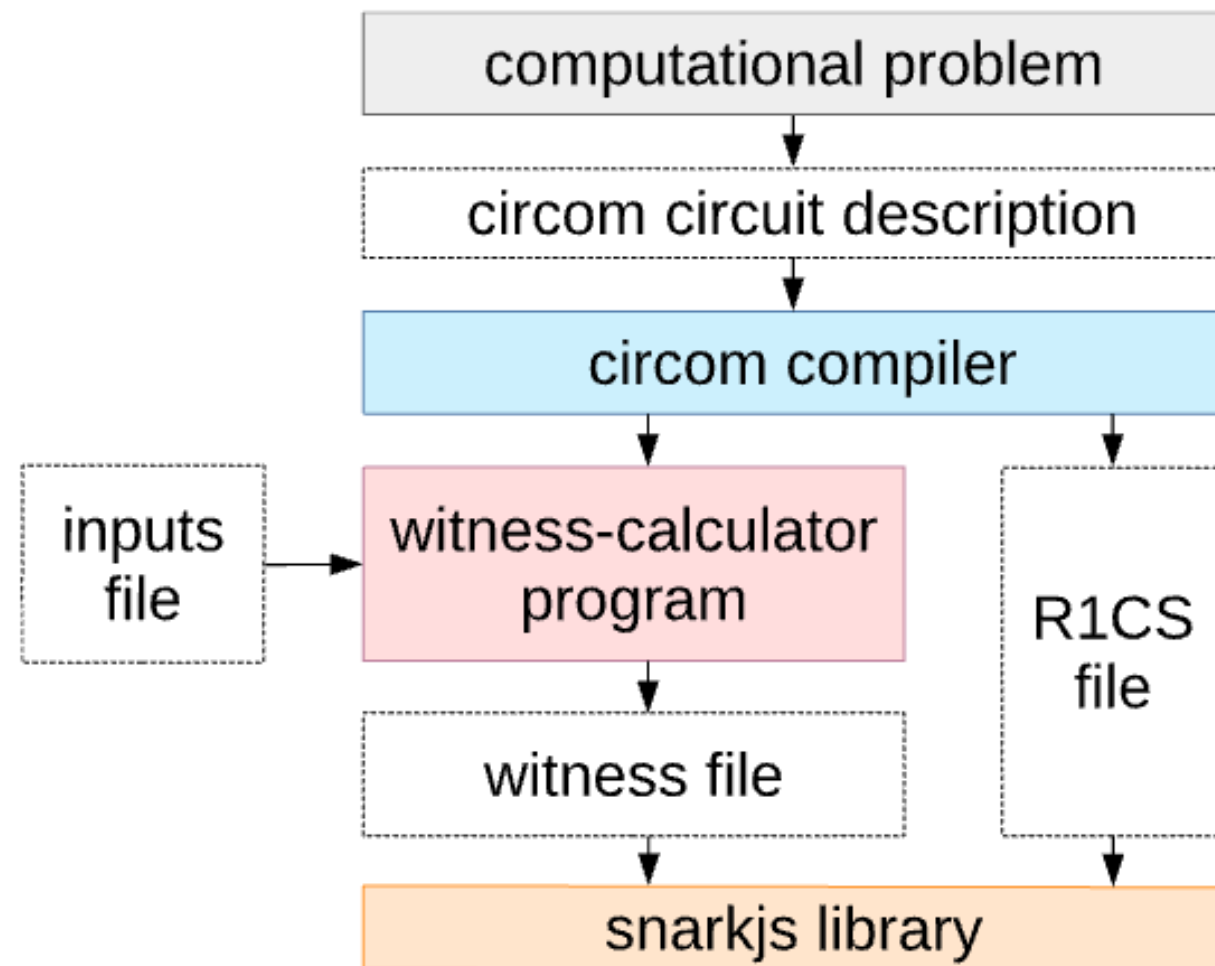
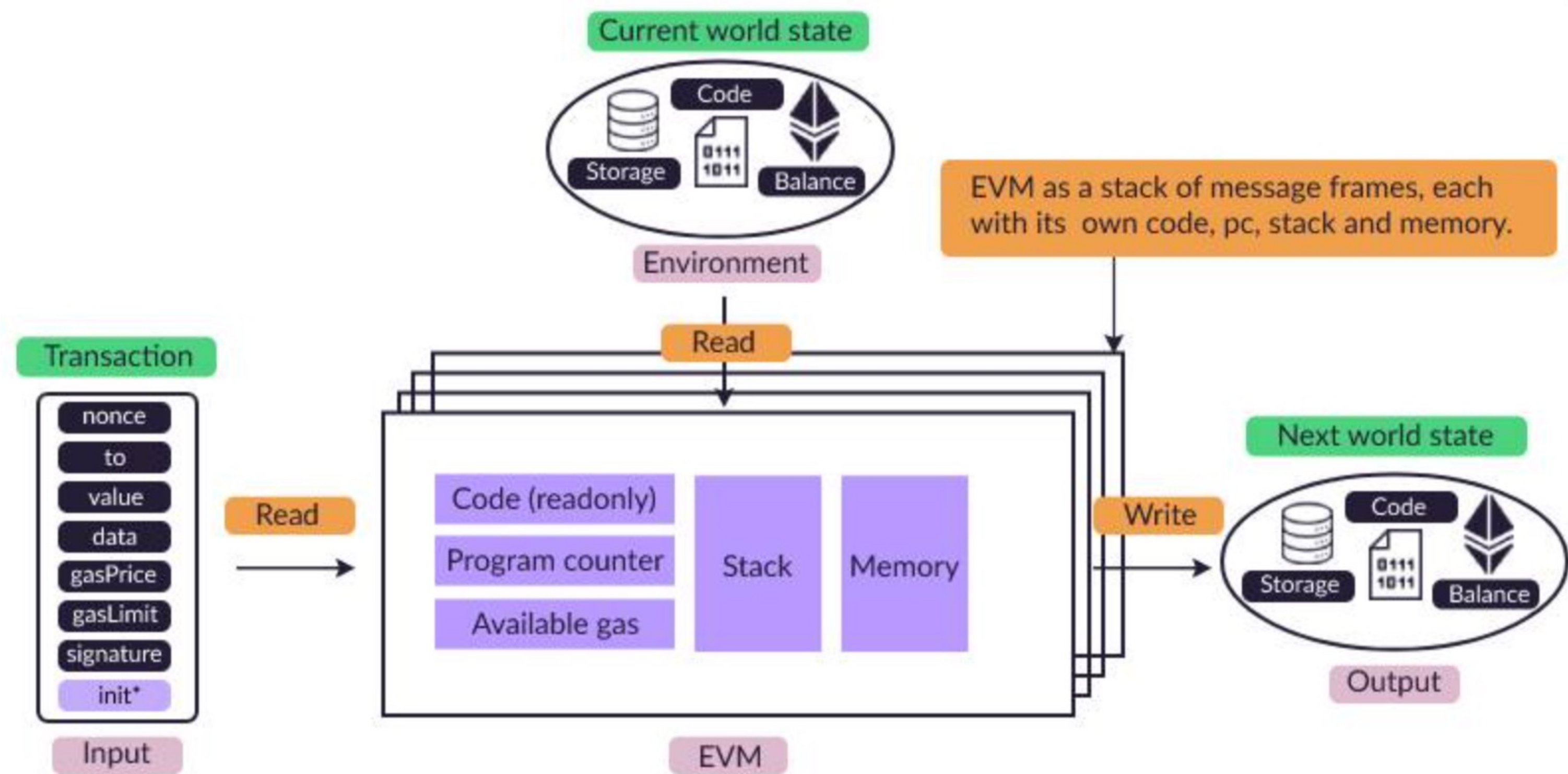


Fig. 2. Classification of the main existing tools in the ZK-SNARK ecosystem.

1. Computational Problem 정의
2. Circom Circuit Description 작성
3. R1CS, Witness-calculator 생성
4. Grouth16, Plonk 등 ZK Proof 생성

#03
ZK-SNARK

Computational Problem



명제 → Computation Problem (e.g. Transaction → zkEVM)

Computational Problem

“특정 Issuer에게 발급받은, 위조되지 않았으며, 나이가 18살 이상(자격 검증)인 VC 소유” ->

$$SMT.InclusionProof(root, key_{age}, value_{age}) = True$$

$$VerifySign(pk_I, root, \sigma_I) = True$$

$$value_{age} \geq 18$$

Verify Sign : Data Integrity 증명을 위한 Eddsa 서명 검증

SMT.InclusionProof & value >= 18 : Selective Disclosure 및 자격 검증(성인 인증)

-> Circom (Constraint DSL) 을 통해 Circuit 구현

Circom Circuit Description

```
// SMT Inclusion Components for 'age'
signal input enabled_age;
signal input siblings_age[nLevels];
signal input oldKey_age;
signal input oldValue_age;
signal input isOld0_age;
signal input key_age;
signal input value_age;
signal input fnc_age;

component smtVerifierAge = SMTVerifier(nLevels);
smtVerifierAge.enabled <== enabled_age;
smtVerifierAge.root <== root;
smtVerifierAge.siblings <== siblings_age;
smtVerifierAge.oldKey <== oldKey_age;
smtVerifierAge.oldValue <== oldValue_age;
smtVerifierAge.isOld0 <== isOld0_age;
smtVerifierAge.key <== key_age;
smtVerifierAge.value <== value_age;
smtVerifierAge.fnc <== fnc_age;
```

```
// EdDSA Verification Component
signal input enabled_eddsa;
signal input Ax;
signal input Ay;
signal input R8x;
signal input R8y;
signal input S;
signal input M; // Use root hash as message

component eddsaVerifier = EdDSAPoseidonVerifier();
eddsaVerifier.enabled <== enabled_eddsa;
eddsaVerifier.Ax <== Ax;
eddsaVerifier.Ay <== Ay;
eddsaVerifier.R8x <== R8x;
eddsaVerifier.R8y <== R8y;
eddsaVerifier.S <== S;
eddsaVerifier.M <== M;
```

```
component ageCheck = GreaterEqThan(32);
ageCheck.in[0] <== value_age; // Age value from SMT proof
ageCheck.in[1] <== 25;
ageCheck.out == 1;
```

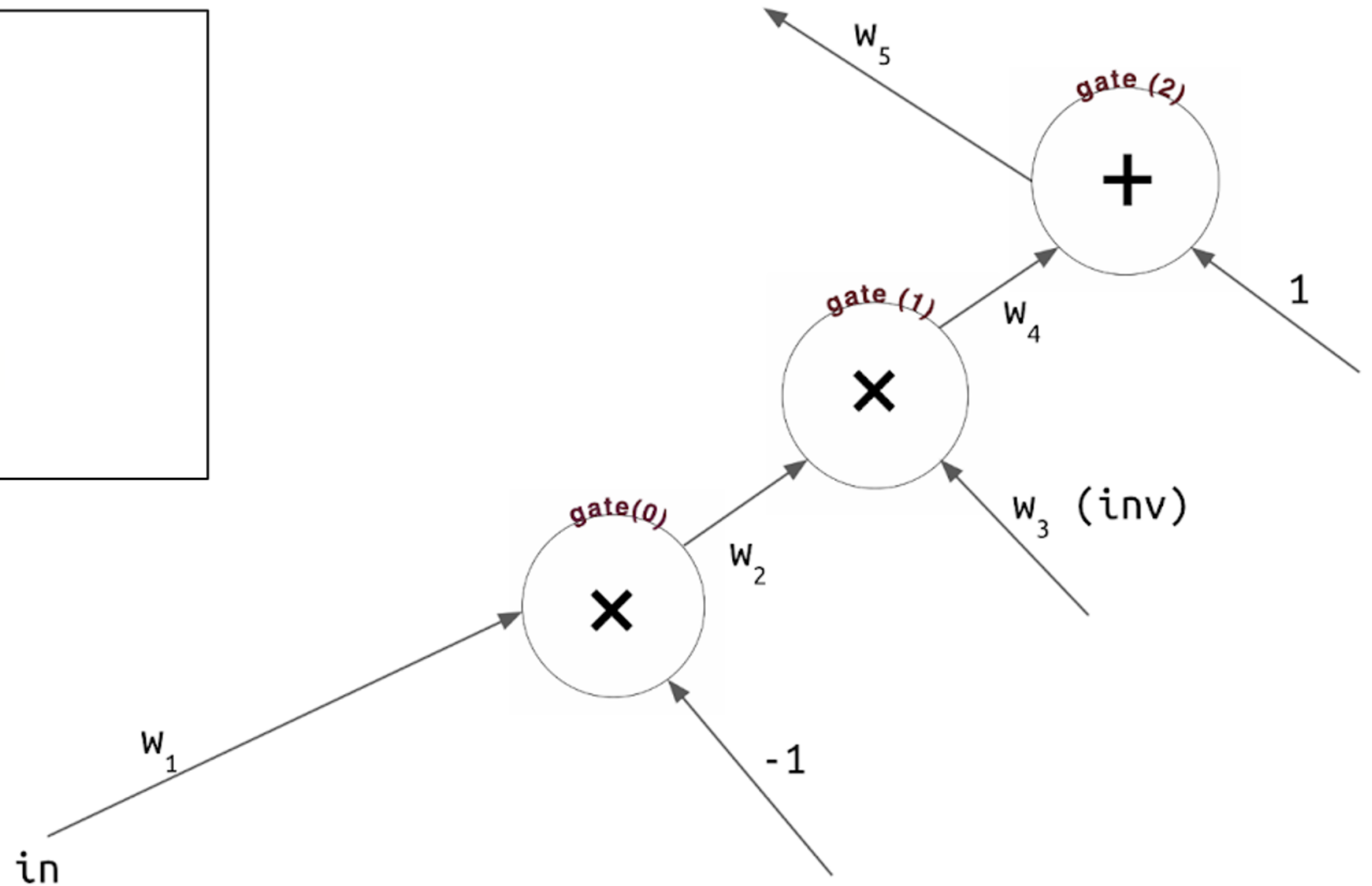
Computation → Arithmetic circuit → R1CS

```
template IsZero() {  
  signal input in ;  
  signal output out;  
  signal inv;  
  inv <-- in != 0 ? 1 / in : 0;  
  out <== -1 * in * inv + 1; in * out == 0;  
}
```

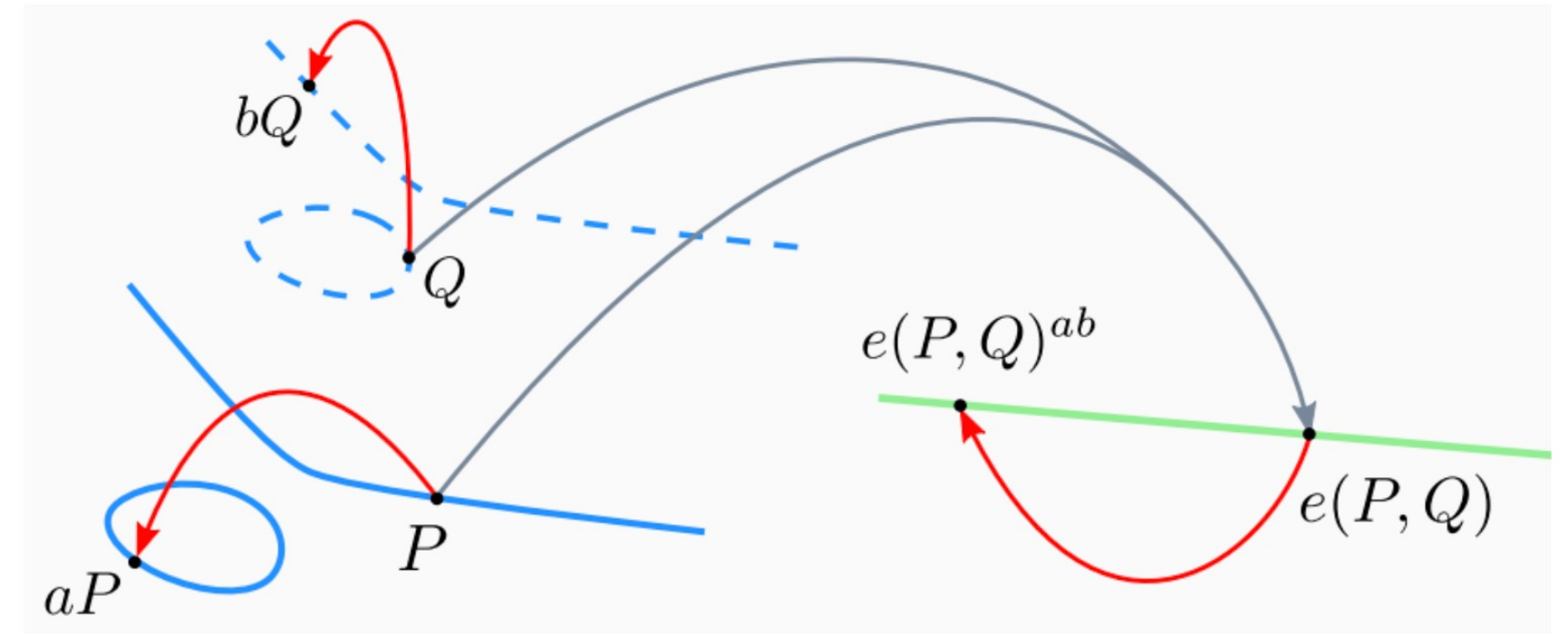
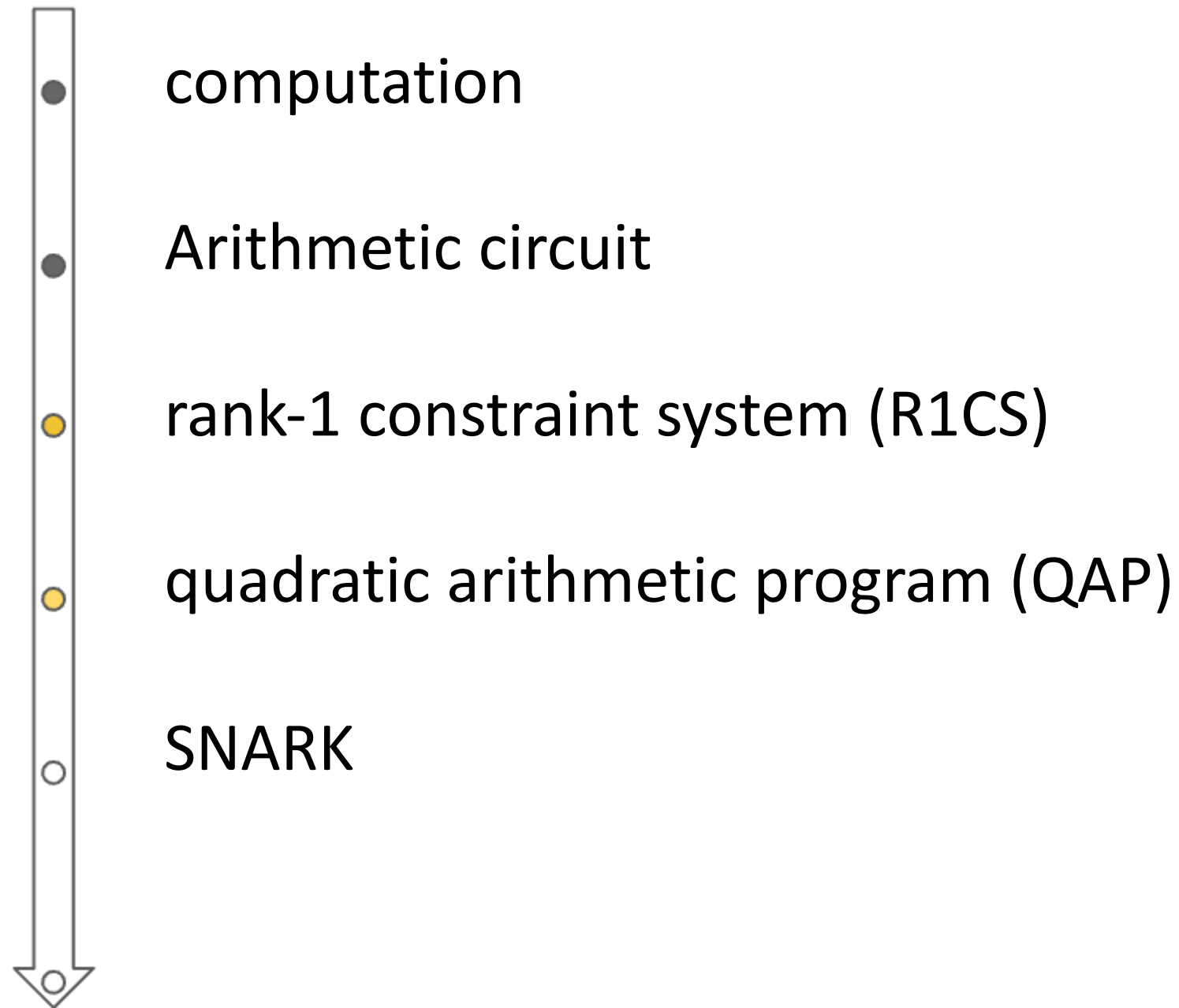
gate 0: $w_1 \cdot (-1) = w_2$

gate 1: $w_2 \cdot w_3 = w_4$

gate 2: $w_4 + 1 = w_5$

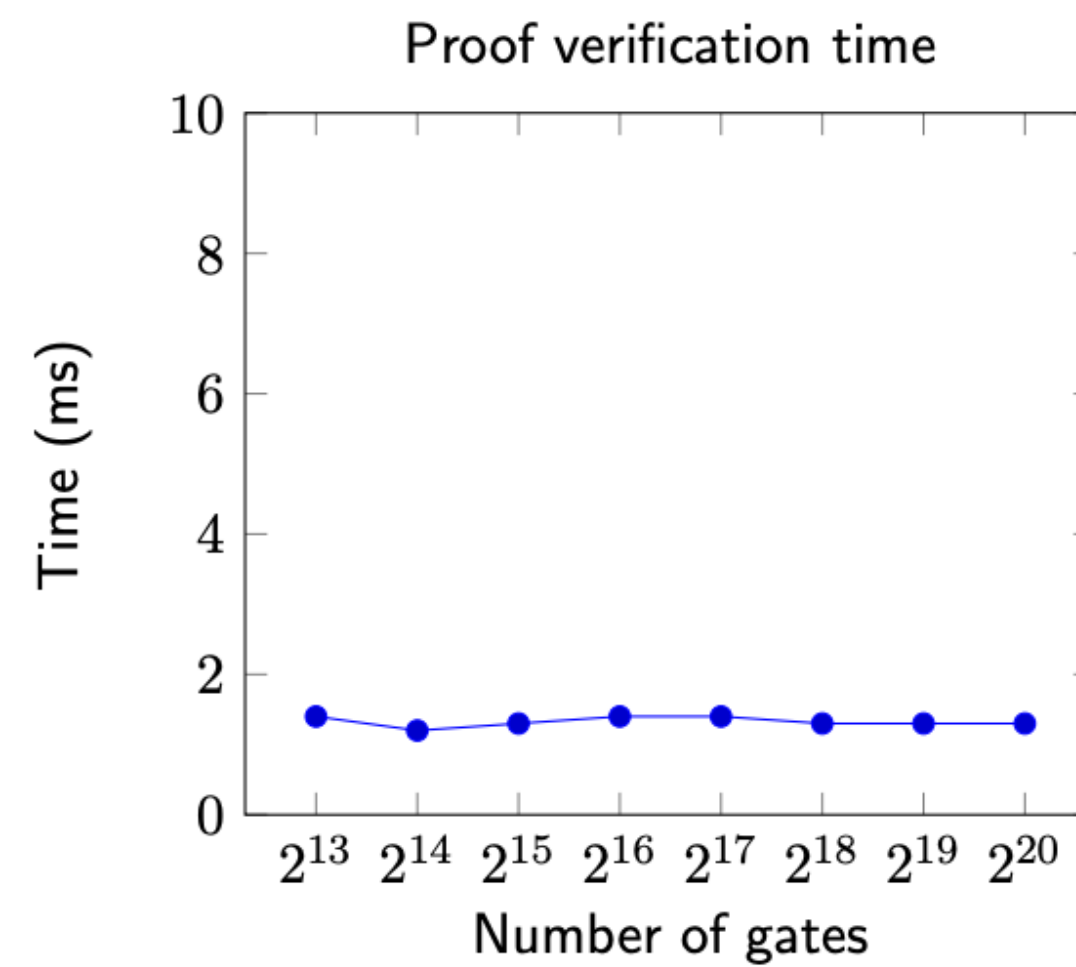
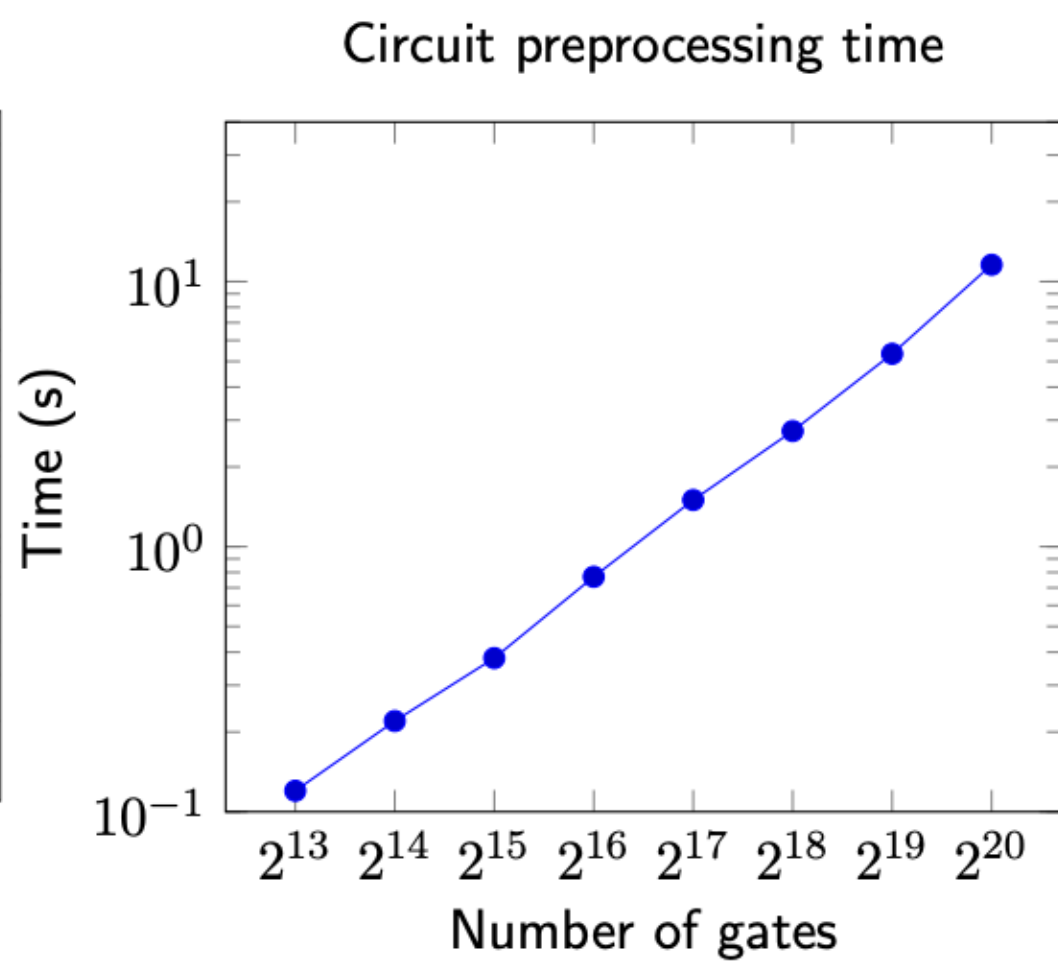
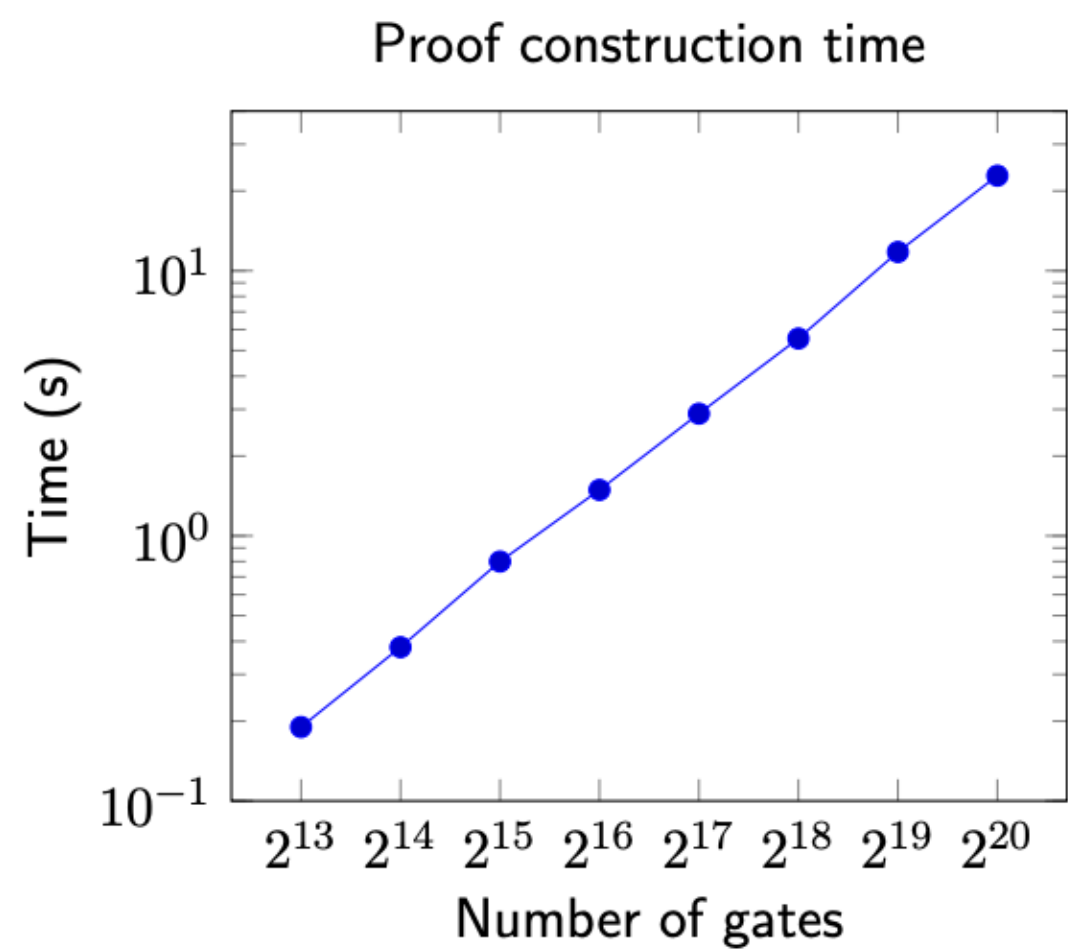


Generating and verifying ZK-SNARK proofs

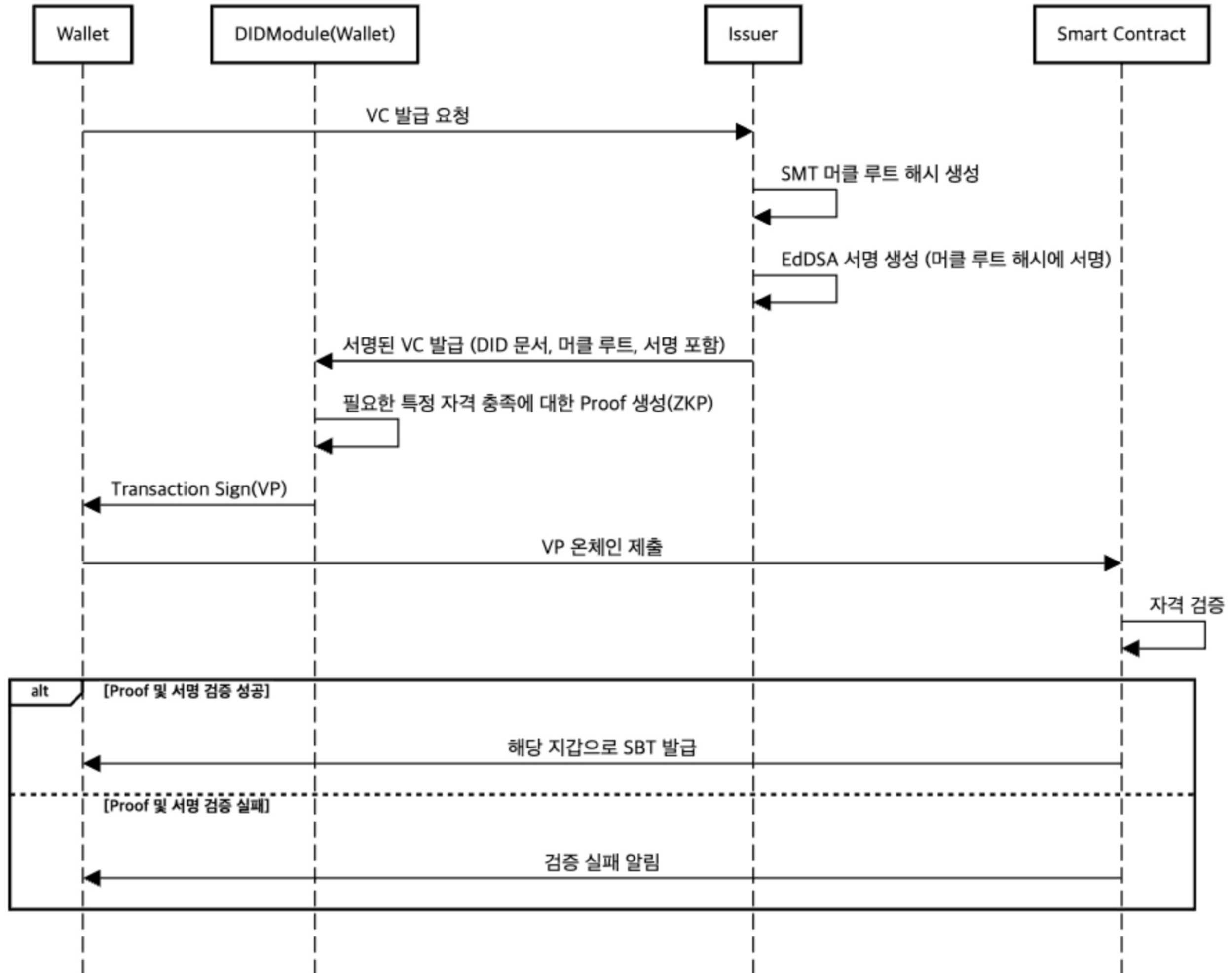


Source: "Pairings are not dead, just resting" by Diego Aranha [ECC-2017](#) (inspired by Avanzi's talk at [SPEED-2009](#)).

$$e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$$
$$e([a]P, [b]Q) = [ab] e(P, Q)$$



Wallet-Based DID Document Credential Verification Process



SnarkJS Extension Example

Create proof

Successfully fetched circuit.wasm file.

Successfully fetched circuit_final.zkey file.

Successfully fetched verification_key.json file.

```
Proof: {
  "pi_a": [
    "5152019152592477803146355633257786506195431839984266034215402762884806731504",
    "19949396677196266867404062879747288912784402846239880024084229872540756179937",
    "1"
  ],
  "pi_b": [
    [
      "3925041166445761154344943199055983523254477164448398984169399419920819750179",
      "15804919121263172327143069347799761886798327804320238259863111812002336191949"
    ],
    [
      "18018879633817194382169315935877438314924090539297021658526539221657959675779",
      "857010692344295018366745175224594319349985399541656362297812449793232843"
    ],
    [
      "1",
      "0"
    ]
  ],
  "pi_c": [
    "5344521046283089176631909097141910812630263287091619893392978449535601949309",
    "9721033412234129371131124748229048309783817522928285657078276794212268391253",
    "1"
  ],
  "protocol": "groth16",
  "curve": "bn128"
}
```

Result: Proof is valid

```
Public Signals: [
  "2",
  "4",
  "1",
  "13277427435165878497778222415993513565335242147425444199013288855685581939618",
  "13622229784656158136036771217484571176836296686641868549125388198837476602820"
]
```

```
Calldata: "[0x0b62f0924cc4d42501d674f0102b6fd34b7d45b1a8f3041027c00fa1a5d0c6fa 0x2a1a558035~"
```

```
contract CNUNFT is ERC721URIStorage, Ownable {
  Groth16Verifier public verifier;
  uint256 public nextTokenId = 1;

  // 이미 발급받았는지 체크하는 매핑
  mapping(address => bool) public hasMinted;

  event NFTMinted(address indexed recipient, uint256 tokenId);

  // 수정된 생성자, Ownable의 생성자에 msg.sender 전달
  constructor(address _verifierAddress) ERC721("CNU NFT", "CNU") Ownable(msg.sender) {
    verifier = Groth16Verifier(_verifierAddress);
  }

  // NFT 발급 함수 (zk-SNARK 프루프 검증을 통과해야 발급 가능)
  function mintNFT(
    uint[2] calldata _pA,
    uint[2][2] calldata _pB,
    uint[2] calldata _pC,
    uint[5] calldata _pubSignals,
    string memory tokenURI
  ) public {
    require(!hasMinted[msg.sender], "NFT already minted");

    // zk-SNARK proof 검증
    require(verifier.verifyProof(_pA, _pB, _pC, _pubSignals), "Invalid proof");

    // NFT 발급 및 메타데이터 설정
    uint256 tokenId = nextTokenId;
    _safeMint(msg.sender, tokenId);
    _setTokenURI(tokenId, tokenURI); // 메타데이터 URI 설정

    hasMinted[msg.sender] = true;
    nextTokenId++;

    emit NFTMinted(msg.sender, tokenId);
  }
}
```

Calldata:

Comma-separated calldata

Contract Address:

0xAb28F873CBb424BF2A6412Cad3d32b9581F4a57d

Contract ABI:

```
[
  {
    "inputs": [
      {
        "internalType": "uint256[2]",
        "name": "_pA",
        "type": "uint256[2]"
      },
      {
        "internalType": "uint256[2][2]",
        "name": "nB"
```

자격 검증

SBT 발급 컨트랙트 Address:

0x9a810c5603338e49fF033204778f90665B7324f8

SBT 발급 컨트랙트 ABI:

```
[
  {
    "inputs": [
      {
        "internalType": "uint256[2]",
        "name": "_pA",
        "type": "uint256[2]"
      },
      {
        "internalType": "uint256[2][2]",
        "name": "nB"
```

SBT 발급

H

Account 1

0xa0DBD...16CDD



CNU KYC NFT

충남대학교 KYC 인증 NFT입니다.

계약 주소

0xA06BF...09D72

토큰 ID

1

토큰 표준

ERC721

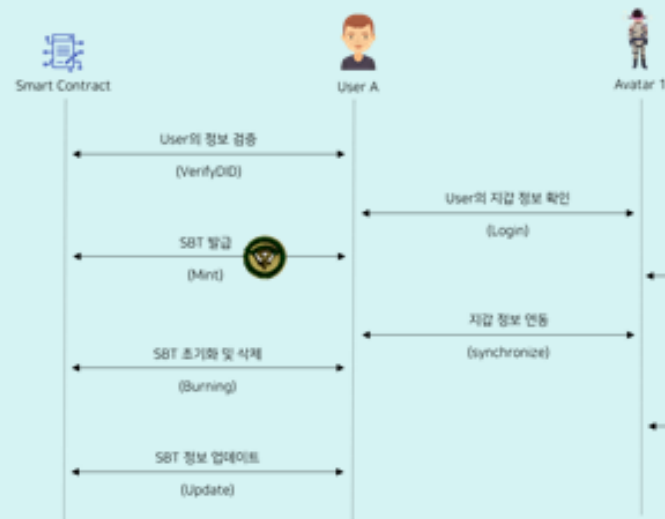
면책 조항: MetaMask는 소스 URL에서 미디어 파일을 가져옵니다.

보내기

1차년도

- 아바타 인증과 권한 관리를 위한 신원 체계 구축

- 메타버스 아바타 신원 규격 정의



ERC-7574 : Authentication SBT using Credential

- ▶ 아바타 인증을 위한 Presentation struct 정의
- ▶ SBT 토큰 발행 및 검증 스마트 컨트랙트 포맷 정의

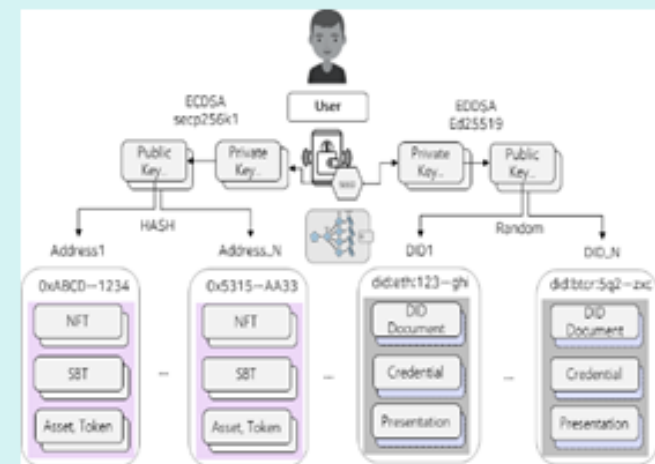
```
// SPDX-License-Identifier: CCB-1.0
pragma solidity ^0.8.0; interface IERC5192 {
    /// @ Function to verify the user's proof.
    /// @ Generated through Zokrates, required parameters
    /// @ Other information such as issuer/holder
    function verify(Proof memory proof, uint[2] memory info) returns (bool);
    /// @ Only those who have completed verifyProof
    /// @ Only the service provider can call the createSBT
    /// @ Issues SBT to authenticated users, with function createSBT(address user, String memory tokenURI) public onlyOwner returns(SBT);
    /// @ Only the service provider can call
    /// @ Issuing an SBT allows cancellation
    function updateTokenURI(address user, string memory tokenURI) public onlyOwner returns(bool);
    /// @ Added cannotTransfer functionality
    function safeTransferFrom(address _from, address _to, uint256 _tokenId) public onlyOwner returns(bool);
}
```

2차년도

- 아바타 신원 및 SBT를 위한 통합 지갑 개발

- DID와 SBT를 관리할 수 있는 통합 지갑

- W3C의 Universal Wallet 2020 요구사항 반영



- 하나의 마스터 시드(SEED)를 통한 HD 통합 지갑 개발

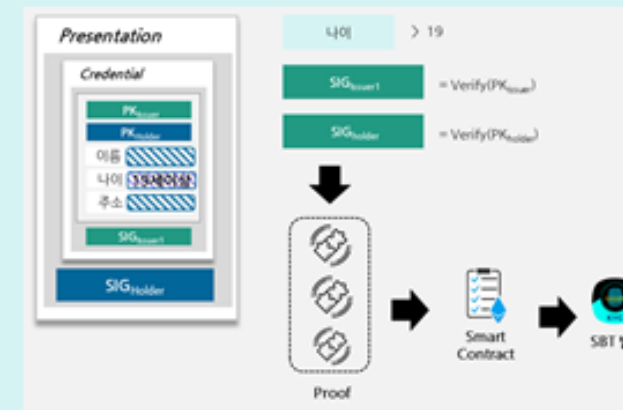
- DID를 관리 하기 위해 EDDSA의 Ed25519 curve를 활용하여 PK 생성, Credential 보관

- SBT를 관리하기 위해 ECDSA의 secp256k1 curve를 활용하여 PK 생성, NFT 및 SBT 보관

3차년도

- SBT 발급 및 DID 검증 스마트 컨트랙트 개발

- DID 검증과 SBT 발급을 스마트 컨트랙트를 통해 진행



- 사용자의 개인정보 보호를 위해 영지식 증명이 적용된 스마트 컨트랙트 개발

- ZK-snarks 기반 영지식증명 알고리즘인 Pinocchio를 활용, 기존 Zokrates 도구를 변형하여 검증 스마트 컨트랙트 생성

- 사용자의 DID를 입력으로 받아, 사용자의 정보에 대한 Proof를 생성한 이후 스마트 컨트랙트를 통해 검증 하여 SBT 발급

4차년도

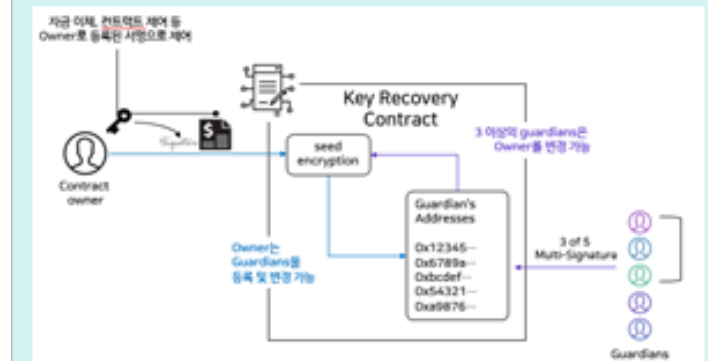
- 메타버스에서의 아바타 권한 관리 실증 서비스 구현

- 메타버스 아바타 매핑(실증서비스)

- 신원 인증 체계, 통합 관리지갑, 스마트 컨트랙트 생성 도구를 이용한 아바타 신원 인증 서비스 구현

- SBT 안전성 평가, 스마트 컨트랙트 취약점 및 실행 비용 최적화 방안 연구

- 사회적 복구(Social Recovery)활용 방안



- 사용자가 SBT 및 DID가 들어있는 지갑의 개인키를 잃어버렸을 경우, 키 변경 스마트 컨트랙트를 활용하여 새로 생성된 키로 변경함으로써 지갑의 소유권 복구가 가능

THANK YOU ®

